TALLER DOCKER

Contenerización, Seguridad de Runtime y Estrés de cgroups

- **Proyecto:** eco-air-backend.
- Individual o Parejas.
- **Entorno:** Terminal con Docker Engine activo sobre Linux, macOS o WSL2.
- **Mecanismo:**
    1. No se califica la sintaxis del archivo; se auditan las firmas SHA-256 de las capas, los UUIDs generados por el daemon local y los logs cronometrados con la marca de tiempo de la máquina.
    2. Cada compilación debe incluir obligatoriamente el código ID del estudiante inyectado en tiempo de construcción.

Caso

El backend eco-air-backend procesa alertas ambientales críticas. Si el contenedor se ejecuta con privilegios de root o si la JVM intenta acaparar toda la memoria RAM del servidor de la estación meteorológica, el proceso será eliminado por el sistema operativo.

Debes encapsular el artefacto en una imagen de producción ligera, segura y con cuotas estrictas de hardware antes de integrarlo a la orquestación multicapa.

RETOS

Reto 1: Construcción Multietapa con Huella Estricta

Se debe crear un archivo Dockerfile y un .dockerignore en la raíz de eco-air-backend cumpliendo estas restricciones:

- **Etapa 1 (builder):** Utilizar maven:3.9.6-eclipse-temurin-17-alpine.
    - Aislar la descarga del pom.xml y ejecutar mvn dependency:go-offline -B antes de transferir src/ para preservar el caché entre builds de código.
    - Declarar un argumento de compilación: ARG STUDENT_ID.
    - Empaquetar el ejecutable .jar omitiendo pruebas.
- **Etapa 2 (runtime):** Utilizar eclipse-temurin:17-jre-alpine.
    - Crear un usuario del sistema sin privilegios: sensor_user con UID 10001 asignado al grupo sensorgroup.
    - Copiar únicamente el archivo .jar resultante y cambiar la titularidad del directorio /app a dicho usuario.
    - Configurar variables de entorno ergonómicas para la JVM
    - Configurar la variable ENV OPERATOR_ID=${STUDENT_ID}.
    - Cambiar al usuario sensor_user antes del ENTRYPOINT.

Comando de construcción obligatorio:

Bash
docker build --build-arg STUDENT_ID="TU_ID_AQUI" -t eco-air-backend:1.0.0

Reto 2: Auditoría Forense y Comprobación de Vulnerabilidades

No debe redactar teoría; debe ejecutar los comandos de diagnóstico en su terminal y consignar las salidas exactas en su reporte:

1. Verificación del Desacoplamiento de Capas

Ejecuta la inspección de historial de la imagen:

Bash
docker history --human --format "table {{.Size}}\t{{.CreatedBy}}"
eco-air-backend:1.0.0

2. Inspección del Proceso del Host (Aislamiento de Privilegios)

Inicia el contenedor en modo detached:

Bash
docker run -d --name eco_air_active -p 8080:8080 eco-air-backend:1.0.0

Consulta la tabla de procesos internos:

Bash
docker top eco_air_active -o pid,user,comm

- **Evidencia:** Demostrar que el comando java no se ejecuta como root, sino bajo el identificador de sensor_user.

docker top eco_air_active -o pid,user,comm

PID	USER	COMMAND
957	10001	java

3. Test de Penetración: Comprobación de Inmutabilidad de Runtime

Intenta escribir un script malicioso en un directorio del sistema:

Bash
docker exec eco_air_active touch /etc/crontab_fake

- **Evidencia:** Colocar evidencia

Comando: docker exec eco_air_active touch /etc/crontab_fake

resultado: touch: cannot touch '/etc/crontab_fake': Permission denied

4. Auditoría de Artefactos de Desarrollo

Bash
docker exec eco_air_active which mvn
docker exec eco_air_active find / -name "*.java" 2>/dev/null

Ambos comandos devolvieron salida vacía, lo que confirma que la imagen final no contiene Maven ni código fuente. Gracias al build multietapa, esas herramientas quedaron en la etapa `builder`, que se descarta. Esto reduce el tamaño de la imagen y la superficie de ataque.

- **Evidencia:** Ambas salidas y explicar el resultado

Reto 3: Calibración de cgroups y Captura del Asesino OOM

Los sensores ambientales envían ráfagas continuas de métricas. En este ejercicio se someterá el contenedor a asfixia de memoria para validar el comportamiento del kernel.

1. Inducción del Colapso por Memoria (*OOMKilled*)

Ejecuta una instancia asignándole un límite de memoria de tan solo **48 MB** (insuficiente para levantar el framework Spring Boot con Hibernate):

Bash
docker run -d --name eco_air_crash -m 48m eco-air-backend:1.0.0

Espera 10 segundos y extrae el estado de terminación del contenedor:

Bash
docker inspect eco_air_crash --format='Status: {{.State.Status}} | ExitCode: {{.State.ExitCode}} | OOMKilled: {{.State.OOMKilled}}'

- **Evidencia obligatoria:** Adjuntar la salida de la consola explicar resultado.

docker inspect eco_air_crash --format='Status: {{.State.Status}} | ExitCode: {{.State.ExitCode}} | OOMKilled: {{.State.OOMKilled}}'

Status: exited | ExitCode: 137 | OOMKilled: true

Campo	Valor	Significado
`Status`	`exited`	El contenedor ya no está corriendo.
`ExitCode`	`137`	Es 128 + 9. El 9 es la señal **SIGKILL**, una orden de muerte inmediata que el proceso no puede ignorar.
`OOMKilled`	`true`	Docker confirma que la causa fue **Out Of Memory**: el proceso superó el límite de memoria.

2. Calibración del Límite Operativo Mínimo

Modifica el parámetro -m de forma iterativa (ej. 256m, 384m, 512m) hasta hallar el umbral mínimo exacto en el que:

1. El contenedor inicie correctamente.
2. Los logs confirmen que la aplicación arrancó en menos de 15 segundos:

   Bash

   docker logs --timestamps eco_air_active | grep "Started EcoAirApplication"

3. El profiling de memoria en caliente se mantenga estable:

Bash

docker stats --no-stream eco_air_active

- **Consigna:** Adjunta la tabla resultante de docker stats indicando: % de CPU, MEM USAGE / LIMIT y MEM %.

```
| CPU % | MEM USAGE / LIMIT | MEM % |

|---|---|---|

| 0.22% | 134.3MiB / 256MiB | 52.47% |
```

FORMATO DE ENTREGA

El estudiante debe subir un archivo en pdf con la justificación del taller y un repositorio o comprimido taller_docker_semana13.zip con la siguiente estructura:

Plaintext
taller-docker-semana13/
├── Dockerfile
├── .dockerignore
├── audit_session.log # Registro crudo de terminal (generado con 'script' o redirect)
└── FORENSIC_REPORT.md # Tabla de evidencias con comandos, hashes y

Contenido obligatorio en FORENSIC_REPORT.md:

- **Tabla de Identidad:**
  - Hash de la imagen: docker inspect --format='{{.Id}}' eco-air-backend:1.0.0
  - Variable de entorno inyectada: docker exec eco_air_active env | grep OPERATOR_ID
  - Consumo final calibrado: salida completa de docker stats --no-stream.